🚀 MASTER PLAN: LSI DOCUMENT AUTOMATION SYSTEM

🧠 PHASE 0 — THINK LIKE A SYSTEM ARCHITECT (CRITICAL)

Before coding anything, you must understand:

This is NOT:

- A CRUD app
- A simple upload/download system

This IS:

- AI-powered **document diff + merge engine**
- Workflow + version-controlled system
- Scalable async processing pipeline

🏗️ PHASE 1 — SYSTEM ARCHITECTURE DESIGN

🔷 Final Architecture (Enterprise Level)

```
Frontend (Next.js)
      ↓
API Gateway (Django / FastAPI)
      ↓
Task Queue (Celery + Redis)
      ↓
Processing Engine
  ├── Document Parser
```

```
  ├── AI Agent (Vertex AI)
  ├── Merge Engine
      ↓
Storage Layer (GCS / Local S3)
      ↓
Database (PostgreSQL)
```

🔷 Key Design Decisions (VERY IMPORTANT)

Component	Choice	Why
Backend	Django + DRF	Built-in auth + admin + scalable
Async	Celery + Redis	AI tasks are heavy
Storage	GCS / S3	Large files
DB	PostgreSQL	Structured tracking
AI	Vertex AI (Gemini)	Large context
Parsing	Python libs	docx, pdf
Frontend	Next.js	dashboard

📁 PHASE 2 — PROJECT SETUP (FOUNDATION)

🔷 Folder Structure (PRODUCTION READY)

lsi-document-automation/

```
├── backend/
│   ├── core/
│   ├── apps/
│   │   ├── users/
│   │   ├── documents/
│   │   ├── processing/
│   │   ├── workflows/
```

```
│   │
│   ├── services/
│   │   ├── storage/
│   │   ├── parser/
│   │   ├── ai/
│   │   ├── merge/
│   │
│   ├── config/
│   ├── manage.py
│
├── frontend/
│   ├── app/
│   ├── components/
│   ├── services/
│
├── infra/
│   ├── docker/
│   ├── terraform/
│
├── docs/
└── README.md
```

🔷 Initial Setup Steps

Backend

```
django-admin startproject core
pip install djangorestframework celery redis psycopg2
```

Frontend

```
npx create-next-app frontend
```

Dev Infra

```
docker-compose up -d
```

🧱 PHASE 3 — DATABASE DESIGN (CRITICAL)

🔷 Core Tables

1. Users

- id
- role (admin, reviewer)

2. Documents

- id
- type (MAIN / SOURCE)
- file_url
- uploaded_by
- version

3. Jobs (IMPORTANT)

- id
- main_document_id
- status (pending, processing, done, failed)
- result_file
- logs

4. Changes (AI Output)

- id
- job_id
- section
- old_text
- new_text

⚙️ PHASE 4 — FILE HANDLING SYSTEM

🔷 Storage Layer

Implement:

upload_file(file)
get_file(file_id)
generate_signed_url()

Support:

- Local (dev)
- GCS (prod)

🔷 Document Parsing Layer

You MUST standardize everything into text.

Format	Tool
PDF	PyMuPDF / pdfplumber
DOCX	python-docx

Output:

```
{
 "sections": [
   { "title": "...", "content": "..." }
 ]
}
```

🤖 PHASE 5 — AI ENGINE (CORE BRAIN)

This is where most projects FAIL.

🔷 Step 1: Chunking Strategy

You CANNOT send 200 pages directly.

Split into:

- Sections
- Paragraph blocks

🔷 Step 2: AI Tasks

Task A — Extract Changes

Input:

- Source files

Output:

```
[
 {
   "section": "Financial Overview",
   "change": "Replace X with Y"
 }
]
```

Task B — Locate in Main Doc

- Semantic matching (embedding or LLM)

Task C — Apply Changes

- Rewrite only affected sections

🔷 Prompt Engineering (VERY IMPORTANT)

You must control AI like this:

```
SYSTEM:
You are a document update engine.

RULES:
- Do NOT change unrelated text
- Maintain formatting
- Apply only explicit updates

INPUT:
MAIN DOC SECTION: ...
```

SOURCE CHANGE: ...

OUTPUT:
UPDATED SECTION ONLY

🔁 PHASE 6 — MERGE ENGINE

After AI updates sections:

- Rebuild document
- Maintain:
  - headings
  - numbering
  - formatting

Output:

- `.docx` (preferred)
- `.pdf`

🔄 PHASE 7 — ASYNC PROCESSING (MANDATORY)

🔷 Why?

AI processing = slow (10–60 sec)

🔷 Flow

User क्लिक "Update"
→ API creates Job
→ Celery processes
→ Status updates
→ Frontend polls

🖥️ PHASE 8 — FRONTEND (NEXT.JS DASHBOARD)

🔷 Pages

1. Login

2. Dashboard

- Upload MAIN
- Upload SOURCE
- Click UPDATE

3. Job Status

- Progress bar
- Logs

4. Download Page

🔐 PHASE 9 — AUTH + SECURITY

- JWT Authentication
- Role-based access
- File access control

📊 PHASE 10 — MONITORING + LOGGING

Use:

- Sentry (errors)
- Logging middleware
- Job logs

🧪 PHASE 11 — TESTING STRATEGY

🔷 Must Test:

- File upload
- Parsing accuracy
- AI output correctness
- Merge consistency

🚀 PHASE 12 — DEPLOYMENT (PRODUCTION)

🔷 Infra

- Backend → Docker + GCP VM / Cloud Run
- Frontend → Vercel
- DB → Cloud SQL
- Storage → GCS

🔷 CI/CD

- GitHub Actions:
  - Test
  - Build
  - Deploy

⚠️ PHASE 13 — REAL WORLD CHALLENGES (IMPORTANT)

From your doc :

🔴 Problem 1: Huge Context

Solution:

- Chunking + memory strategy

🔴 Problem 2: Formatting Loss

Solution:

- Structured parsing (NOT plain text only)

🔴 Problem 3: Wrong AI Edits

Solution:

- Human review mode (optional future)

📈 PHASE 14 — FUTURE SCALING

- Version control (like Git for documents)
- Diff viewer UI
- Multi-user workflow
- Approval system (like your helpdesk system idea)

🧭 FINAL BUILD ORDER (VERY IMPORTANT)

Follow EXACTLY this:

STEP 1

Backend basic setup (Django + DB)

STEP 2

File upload + storage

STEP 3

Parsing engine

STEP 4

Basic AI integration (simple test)

STEP 5

Job system (Celery)

STEP 6

Full pipeline (upload → process → output)

STEP 7

Frontend dashboard

STEP 8

Polish + error handling

STEP 9

Deployment